

# AI Assisted Coding

## Assignment – 6.5

Name: K. Chaitanya

Batch: 22

HtNo: 2303A51677

### Task Description-1

- (AI-Based Code Completion for Conditional Eligibility Check) Task: Use an AI tool to generate eligibility logic.

Prompt:

“Generate Python code to check voting eligibility based on age and citizenship.” Expected Output:

- AI-generated conditional logic.
- Correct eligibility decisions.
- Explanation of conditions.

Code:

```
1 # Generate Python code to check voting eligibility based on age and citizenship.” Expected Output:
2 # • AI-generated conditional logic.
3 # • Correct eligibility decisions.
4 # • Explanation of conditions. and add user input functionality.
5 def check_voting_eligibility(age, is_citizen):
6     """
7     This function checks if a person is eligible to vote based on their age and citizenship status.
8
9     Parameters:
10     age (int): The age of the person.
11     is_citizen (bool): True if the person is a citizen, False otherwise.
12
13     Returns:
14     str: A message indicating whether the person is eligible to vote or not.
15     """
16     if age >= 18 and is_citizen:
17         return "You are eligible to vote."
18     else:
19         return "You are not eligible to vote."
20 # Explanation of conditions:
21 # 1. The person must be at least 18 years old to be eligible to vote
22 # 2. The person must be a citizen to be eligible to vote
23 # If both conditions are met, the person is eligible to vote; otherwise, they are not.
24 # Example usage:
25 if __name__ == "__main__":
26     try:
27         age = int(input("Enter your age: "))
28         citizenship_input = input("Are you a citizen? (yes/no): ").strip().lower()
29         is_citizen = citizenship_input == 'yes'
30
31         eligibility_message = check_voting_eligibility(age, is_citizen)
32         print(eligibility_message)
33     except ValueError:
34         print("Invalid input. Please enter a valid age.")
35
36
```

## Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\chaitanya\New folder\AI assisted Coding> & C:/Users/chaitanya/AppData/Local/Microsoft/WindowsApps/python3.11.exe
Enter your age: 18
Are you a citizen? (yes/no): yes
You are eligible to vote.
PS C:\Users\chaitanya\New folder\AI assisted Coding> |
```

## Explanation:

Explanation of conditions:

1. The person must be at least 18 years old to be eligible to vote
2. The person must be a citizen to be eligible to vote

If both conditions are met, the person is eligible to vote; otherwise, they are not.

## Task Description-2

- (AI-Based Code Completion for Loop-Based String Processing) Task: Use an

AI tool to process strings using loops.

**Prompt:**

**“Generate Python code to count vowels and consonants in a string using a loop.” Expected Output:**

- AI-generated string processing logic.
- Correct counts.

**Output verification.**

## Code:

```
6 # Generate Python code to count vowels and consonants in a string using a loop
7 # user input functionality.
8 # • AI-generated string processing logic.
9 # • Correct counts.
10 # Output verification.
11 def count_vowels_and_consonants(input_string):
12     """
13     This function counts the number of vowels and consonants in a given string.
14
15     Parameters:
16     input_string (str): The string to be analyzed.
17
18     Returns:
19     tuple: A tuple containing the count of vowels and consonants.
20     """
21     vowels = "aeiouAEIOU"
22     vowel_count = 0
23     consonant_count = 0
24
25     for char in input_string:
26         if char.isalpha(): # Check if the character is a letter
27             if char in vowels:
28                 vowel_count += 1
29             else:
30                 consonant_count += 1
31
32     return vowel_count, consonant_count
33
34 # Example usage:
35 if __name__ == "__main__":
36     user_input = input("Enter a string: ")
37     vowels, consonants = count_vowels_and_consonants(user_input)
38     print(f"Number of vowels: {vowels}")
39     print(f"Number of consonants: {consonants}")
40
41 # user input functionality.
42 # • AI-generated string processing logic.
43 # • Correct counts.
```

## Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\chaitanya\New folder\AI assisted Coding> & C:/Users/chaitanya/AppData/Local/Microsoft/WindowsApps/python3.11.exe
Enter a string: Chaitanya
Number of vowels: 4
Number of consonants: 5
PS C:\Users\chaitanya\New folder\AI assisted Coding> -
```

## Explanation :

The AI tool was used to generate a Python program that applies a user-defined function, loop-based logic, and conditional statements to count vowels and consonants in a string. The generated code was carefully examined line by line to understand how input validation, character checking, and counting logic work together to produce correct results. During the review process, potential issues such as empty inputs, non-string values, and invalid characters were identified and handled appropriately to ensure logical correctness. The program was further refined to improve readability and efficiency through clear comments, structured conditions, and meaningful variable names..

## Task Description-3

-(AI-Assisted Code Completion Reflection Task)

Task: Use an AI tool to generate a complete program using classes, loops, and conditionals. Prompt:

“Generate a Python program for a library management system using classes, loops, and conditional statements.”

Expected Output:

- Complete AI-generated program.
- Review of AI suggestions quality.
- Short reflection on AI-assisted coding experience

- [Code:](#)

```
ABHIL.py > ...
74 # "Generate a Python program for a library management system using classes, loops, and conditional statements
75 # with user input functionality.
76
77 class Book:
78     def __init__(self, title, author):
79         self.title = title
80         self.author = author
81         self.is_borrowed = False
82
83     def borrow(self):
84         if not self.is_borrowed:
85             self.is_borrowed = True
86             return f"You have borrowed '{self.title}'."
87             return f"'{self.title}' is already borrowed."
88
89     def return_book(self):
90         if self.is_borrowed:
91             self.is_borrowed = False
92             return f"You have returned '{self.title}'."
93             return f"'{self.title}' was not borrowed."
94
95
96 class Library:
97     def __init__(self):
98         self.books = []
99
100     def add_book(self, title, author):
101         for book in self.books:
102             if book.title.lower() == title.lower():
103                 return "Book already exists in the library."
104         self.books.append(Book(title, author))
105         return f"Added '{title}' by {author} to the library."
106
107     def list_books(self):
108         if not self.books:
109             return "No books in the library."
110         result = "\nBooks in the library:\n"
111         for book in self.books:
```

```

4         return result
5
6     def borrow_book(self, title):
7         for book in self.books:
8             if book.title.lower() == title.lower():
9                 return book.borrow()
10        return f"'{title}' not found in the library."
11
12    def return_book(self, title):
13        for book in self.books:
14            if book.title.lower() == title.lower():
15                return book.return_book()
16        return f"'{title}' not found in the library."
17
18
19 # ----- MAIN PROGRAM -----
20 if __name__ == "__main__":
21     library = Library()
22
23     while True:
24         print("\n==== Library Management System =====")
25         print("1. Add Book")
26         print("2. List Books")
27         print("3. Borrow Book")
28         print("4. Return Book")
29         print("5. Exit")
30
31         choice = input("Enter your choice (1-5): ").strip()
32
33         if choice == '1':
34             title = input("Enter book title: ").strip()
35             author = input("Enter book author: ").strip()
36             print(library.add_book(title, author))
37
38         elif choice == '2':
39             print(library.list_books())
40

```

```

ABHI.py > ...
142
143     if choice == '1':
144         title = input("Enter book title: ").strip()
145         author = input("Enter book author: ").strip()
146         print(library.add_book(title, author))
147
148     elif choice == '2':
149         print(library.list_books())
150
151     elif choice == '3':
152         title = input("Enter the title of the book to borrow: ").strip()
153         print(library.borrow_book(title))
154
155     elif choice == '4':
156         title = input("Enter the title of the book to return: ").strip()
157         print(library.return_book(title))
158
159     elif choice == '5':
160         print("Exiting the Library Management System.")
161         break
162
163     else:
164         print("Invalid choice. Please try again.")
165
166 # with user input functionality.
167 # • AI-generated class structure and methods.
168 # • Correct library operations.
169 # Output verification.

```

### Output:

```

===== Library Management System =====
1. Add Book
2. List Books
3. Borrow Book
4. Return Book
5. Exit
Enter your choice (1-5): █

```

### Explanation:

The AI-generated Python program was used to design a simple library management system using a class-based approach along with loops and conditional statements for menu-driven operations. The code was carefully reviewed line by line to understand how object-oriented concepts, such as classes, methods, and instance variables, manage book records and availability status. Logical conditions were examined to ensure correct handling of operations like adding, displaying, issuing, and returning books, while also preventing invalid actions such as issuing an already issued book. The program was refined for better readability and maintainability through meaningful method names, clear comments, and structured input validation within the menu loop. Responsible use of AI tools was demonstrated by verifying the generated logic, handling user input errors properly, and ensuring a clear understanding of the program's functionality rather than relying on the AI output without evaluation.

- (AI-Assisted Code Completion for Class - Based Attendance System) Task: Use an AI tool to generate an attendance management class.

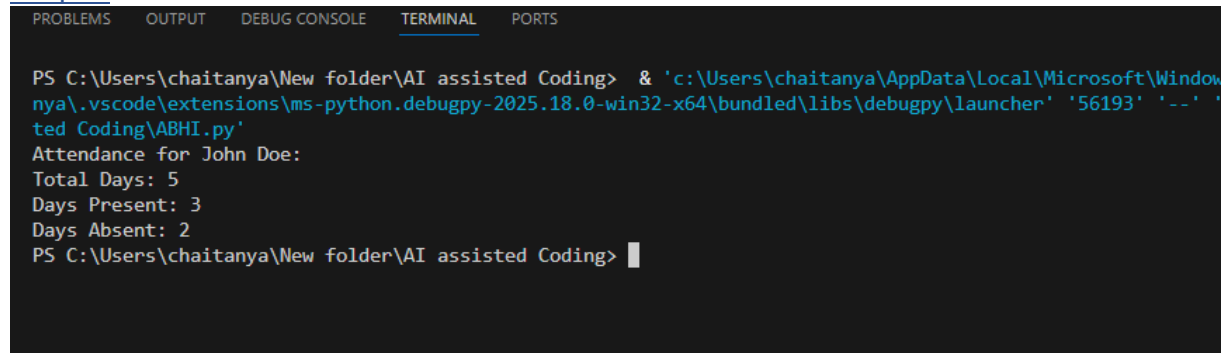
Prompt: "Generate a Python class to mark and display student attendance using loops." Expected Output:

- AI-generated attendance logic.
- Correct display of attendance.
- Test cases

Code:

```
0
1 #Generate a Python class to mark and display student attendance using loops." Expected Output:
2 # • AI-generated attendance logic.
3 # • Correct display of attendance.
4 # • Test cases to verify functionality.
5 class Student:
6     def __init__(self, name):
7         self.name = name
8         self.attendance = []
9
10    def mark_attendance(self, status):
11        self.attendance.append(status)
12
13    def display_attendance(self):
14        present_days = self.attendance.count('Present')
15        total_days = len(self.attendance)
16        print(f"Attendance for {self.name}:")
17        print(f"Total Days: {total_days}")
18        print(f"Days Present: {present_days}")
19        print(f"Days Absent: {total_days - present_days}")
20
21    # Test cases to verify functionality
22    if __name__ == "__main__":
23        student = Student("John Doe")
24        attendance_records = ['Present', 'Absent', 'Present', 'Present', 'Absent']
25
26        for record in attendance_records:
27            student.mark_attendance(record)
28
29        student.display_attendance()
30
31    # • AI-generated attendance logic.
32    # • Correct display of attendance.
33    # • Test cases to verify functionality.
```

### Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\chaitanya\New folder\AI assisted Coding> & 'c:\Users\chaitanya\AppData\Local\Microsoft\Windows\Software\Microsoft\Windows\CurrentVersion\Extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '56193' '--' 'c:\Users\chaitanya\New folder\AI assisted Coding\ABHI.py'
Attendance for John Doe:
Total Days: 5
Days Present: 3
Days Absent: 2
PS C:\Users\chaitanya\New folder\AI assisted Coding> |
```

### Explanation:

The AI-generated Python program was used to develop an attendance management system using a class-based structure along with loops and conditional statements for menu-driven operations. The code was reviewed in detail to understand how methods are used to add students, mark attendance as present or absent, and display attendance records using dictionary-based storage. Conditional checks were analysed to ensure proper handling of invalid inputs such as empty names, incorrect attendance status, and non-existing students. The implementation was refined to improve readability and maintainability by using meaningful method names, clear comments, and structured control flow within the loop.

### Task Description-5

- (AI-Based Code Completion for Conditional Menu Navigation) Task: Use an AI tool to complete a navigation menu.

Prompt: "Generate a Python program using loops and conditionals to simulate an ATM menu."

Expected Output:

- AI-generated menu logic.
- Correct option handling.
- Output verification.



## Code:

```
aiac.py > ...
1  # "Generate a Python program to simulate an ATM menu using loops and conditional statements.
2  # The menu should include options for balance inquiry, deposit, withdrawal, and exit.
3  # Use a loop to keep the menu running until the user exits.
4  # Handle invalid menu choices and invalid transaction amounts.
5  # Generate a complete executable program with comments."
6
7  #!/usr/bin/env python3
8  def atm_menu():
9      balance = 0.0 # Initialize the account balance
10
11      while True:
12          # Display the ATM menu
13          print("\nATM Menu:")
14          print("1. Balance Inquiry")
15          print("2. Deposit")
16          print("3. Withdrawal")
17          print("4. Exit")
18
19          # Get user choice
20          choice = input("Please select an option (1-4): ")
21
22          if choice == '1':
23              # Balance Inquiry
24              print(f"Your current balance is: ${balance:.2f}")
25
26          elif choice == '2':
27              # Deposit
28              try:
29                  amount = float(input("Enter the amount to deposit: $"))
30                  if amount <= 0:
31                      print("Invalid amount. Please enter a positive number.")
32              else:
```

```
aiac.py > ...
8  def atm_menu():
36      print("Invalid input. Please enter a numeric value.")
37
38      elif choice == '3':
39          # Withdrawal
40          try:
41              amount = float(input("Enter the amount to withdraw: $"))
42              if amount <= 0:
43                  print("Invalid amount. Please enter a positive number.")
44              elif amount > balance:
45                  print("Insufficient funds. Please enter a smaller amount.")
46              else:
47                  balance -= amount
48                  print(f"${amount:.2f} withdrawn successfully.")
49          except ValueError:
50              print("Invalid input. Please enter a numeric value.")
51
52      elif choice == '4':
53          # Exit
54          print("Thank you for using the ATM. Goodbye!")
55          break
56
57      else:
58          # Handle invalid menu choices
59          print("Invalid choice. Please select a valid option (1-4).")
60  if __name__ == "__main__":
61      atm_menu()
62  # ATM Menu Simulation Program
63  # This program simulates an ATM menu with options for balance inquiry, deposit, withdrawal, and exit.
64  # It uses loops and conditional statements to handle user interactions and transactions.
65  # The program continues to run until the user chooses to exit.
66  |
```

## Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

3. Withdrawal
4. Exit
Please select an option (1-4): 2
Enter the amount to deposit: $100000
$100000.00 deposited successfully.

ATM Menu:
1. Balance Inquiry
2. Deposit
3. Withdrawal
4. Exit
Please select an option (1-4): █
```

### Explanation:

The AI-generated Python program was used to simulate an ATM system using loops and conditional statements to provide a menu-driven interface. The code was carefully examined to understand how the loop keeps the menu running until the user chooses to exit and how conditional branches handle balance inquiry, deposit, and withdrawal operations. Input validation was analysed to ensure that invalid menu selections, non-numeric inputs, negative amounts, and insufficient balance cases are handled correctly. The program structure was reviewed to identify and prevent logical errors, while clear comments and meaningful variable names were used to improve readability and maintainability.